

SCP Reference Manual

July 13, 2026

SCP-package

SCP: Power Evaluation and Study Design for Circadian Biomarker Detection

Description

SCP (Simulation-based Circadian Power) is a framework for sample-size planning in circadian omics studies, where limited sample availability, uncontrolled collection times, and the multiple-testing burden of genome-wide analysis routinely constrain the design. It calibrates the analysis to a user's pilot dataset, drawing on the pilot's gene-level distributions of amplitude, noise, and phase together with its time-of-day sampling to capture transcriptome-wide signal heterogeneity. From this calibration it estimates FDR-controlled power for two goals: single-cohort detection of rhythmic biomarkers, and two-group differential analysis of rhythmicity, phase, or mesor. A bootstrap layer quantifies the uncertainty that the finite pilot contributes to each power estimate, and the framework accommodates any model-based circadian detector (cosinor or non-cosinor). It bundles roughly 160 curated circadian transcriptomic pilots spanning many tissues and several species, plus a Shiny application for point-and-click power calculation and biomarker exploration.

Main functions

Pilot database • `scp_pilots`

- `scp_load_pilot`
- `scp_pilot_search`

Calibrate your own pilot • `estCircadianParam`

- `estCircadianParam2H`
- `estCircadianParamTwoGroup`
- `prepCircadianData`

Design and options • `CircadianBioOptions`

- `CircadianDesignOptions`
- `CircadianAnalysisOptions`
- `CircadianBootstrapOptions`

Run power analysis • `runSimsSingleCohort`

- `runDifferentialPower`
- `runBootstrapDesignGrid`
- `npower`
- `circaPowerApproxN80`

- [recommendDesign](#)
- Plot results**
 - [plotSingleCohortPower](#)
 - [plotDiffPower](#)
 - [plotBootstrapDesignGrid](#)
- Detect rhythmicity**
 - [detect_cosinor](#)
- Simulate and helpers**
 - [simCircadianSingleCohort2H](#)
 - [makeAdaptiveRStrata](#)
- App**
 - [launchShiny](#)

Author(s)

Maintainer: Thien Quy Pham <quythien14@gmail.com> (University of Pittsburgh)

Authors:

- Hao Wang (University of Pittsburgh)
- Marianne L. Seney (University of Pittsburgh)
- Colleen A. McClung (University of Pittsburgh)
- Guan Yu (University of Pittsburgh)
- George C. Tseng <ctseng@pitt.edu> (University of Pittsburgh)

See Also

Useful links:

- <https://github.com/quythien/SCP>
- Report bugs at <https://github.com/quythien/SCP/issues>

scp_pilot_database *SCP Pilot Dataset Database*

Description

The SCP package bundles a curated database of public circadian transcriptomic pilot datasets that users can plug into `runSimsSingleCohort()` or `runDifferentialPower()` as the parameter source for power simulation. Each bundled pilot is a pre-fit `CircadianBioOptions` object stored under `inst/extdata/pilots/<species>/`, alongside a CSV manifest that documents provenance (accession, citation, license), study design (active vs passive), and condition arm.

Three user-facing functions are exposed:

`scp_pilots` Return the manifest as a `data.frame`, optionally filtered by species and ingestion status.

`scp_load_pilot` Load and return one standardized pilot object given species, dataset, tissue, and condition.

`scp_pilot_search` Convenience search: filter the manifest by free-text query, species, tissue regex, and in-vivo vs cell-line sample type.

 scp_pilots

List Bundled SCP Pilot Datasets

Description

Reads the bundled pilot manifest (`inst/extdata/pilots/manifest.csv`) and returns it as a `data.frame`. Optionally filters by species and ingestion status so a user can quickly enumerate the pilots they actually have available.

Usage

```
scp_pilots(species = NULL, status = "ingested")
```

Arguments

species	Optional character. One or more of "human", "baboon", "other_primate", "mouse", "rat". NULL (default) returns all species.
status	Character. Filter on the status column. Defaults to "ingested" (datasets actually present on disk). Use NULL or "all" to return every row, including "pending", "verify_failed", "controlled", and "ingest_failed" entries.

Value

A `data.frame` with one row per (dataset, tissue, condition) pilot. Columns: species, dataset, tissue, condition, n, ngenes, design, assay, sample_type, accession, citation, license, file, status.

Examples

```
scp_pilots()
scp_pilots(species = "human")
scp_pilots(status = "all")
```

 scp_load_pilot

Load a Bundled SCP Pilot Dataset

Description

The returned object is a `CircadianBioOptions` pilot from the bundled database, produced by the standard cosinor pipeline and ready for power simulation. It has the same form as a pilot you calibrate from your own data, so it can be passed straight to the single-cohort and differential power runners.

Usage

```
scp_load_pilot(
  species,
  dataset,
  tissue,
  condition = NULL,
  alpha_pilot = 0.01,
  adjust = c("none", "BH"),
  K = 1,
  paired_sigma = TRUE
)
```

Arguments

species	Character, length 1. One of "human", "baboon", "other_primate", "mouse", "rat".
dataset	Character, length 1. Manifest dataset field (typically GEO accession or short label, e.g. "GTEx", "GSE160521").
tissue	Character, length 1. Tissue or region label as recorded in the manifest (e.g. "Liver", "NAc").
condition	Character, length 1, optional. Condition arm (e.g. "Control", "All"). If NULL and the (species, dataset, tissue) triple identifies more than one row, the function errors and prints the available conditions.
alpha_pilot	Numeric in $(0, \text{cap}]$. Rhythmicity threshold used to re-derive <code>prop_rhythmic</code> and the amplitude/phase/sigma distributions from the pilot's stored per-gene rhythm table. Default 0.01 matches the build-time threshold. Only takes effect for pilots rebuilt with a <code>rhythm_fit</code> table; older pilots ignore it (with a warning) and return their frozen build-time distributions.
adjust	Multiple-testing adjustment applied to the per-gene p-values before thresholding at <code>alpha_pilot</code> : "none" (raw cosinor p, default) or "BH" (Benjamini-Hochberg q-value).
K	Harmonic order of the requested pilot: 1 (single-harmonic cosinor, default) or 2 (two-harmonic). For $K = 2$ the two-harmonic variant of the pilot (<code><file>_2H.rds</code>) is loaded; an error is raised if no such variant has been built for the pilot.
paired_sigma	Logical (default TRUE). Keep the per-gene (A , σ) pairing so the simulator draws amplitude and noise from the same pilot gene, preserving the realistic narrow $\tilde{r} = A/\sigma$ distribution (the marginal-power / Fig 1-2 Panel A convention). FALSE decouples sigma from amplitude, giving the wide $\tilde{r}$ used by the effect-size-stratified panels (Fig 1-2 Panels B/C).

Value

A `CircadianBioOptions` object with empirical distributions of mesor, log-sigma, amplitude, phase, and proportion rhythmic, ready for downstream simulation.

See Also

[estCircadianParam](#) to build the same object from your own pilot data.

Examples

```

bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All",
                      alpha_pilot = 0.05)
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All",
                      alpha_pilot = 0.1, adjust = "BH")

```

scp_pilot_search *Search the SCP Pilot Database*

Description

Convenience filter over the pilot manifest. Combines free-text substring matching against dataset, tissue, citation, and accession fields with optional species and tissue-regex filters and an in-vivo vs cell-line restriction. Useful when a user wants to pick a pilot whose design (tissue, sample size, signal strength) matches their planned study.

Usage

```

scp_pilot_search(
  query = NULL,
  species = NULL,
  tissue_pattern = NULL,
  sample_type = "in_vivo"
)

```

Arguments

query	Character, optional. Case-insensitive substring matched against dataset, tissue, accession, citation. NULL matches everything.
species	Character, optional. Restrict to these species.
tissue_pattern	Character, optional. Regular expression matched against tissue (case-insensitive).
sample_type	Character. Default "in_vivo". Use NULL to include cell-line pilots as well.

Value

A data.frame: rows of the manifest passing every active filter.

Examples

```

scp_pilot_search("liver")
scp_pilot_search(species = "mouse", tissue_pattern = "brain|cortex")
scp_pilot_search(sample_type = NULL)    # include cell lines

```

estCircadianParam *Estimate Circadian Pilot Parameters (Single-Cohort)*

Description

Fits a cosinor model per gene, ranks rhythmic genes by p-value, and returns empirical parameter distributions packaged as a `CircadianBioOptions` object suitable for downstream simulation and power analysis.

Usage

```
estCircadianParam(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

<code>data</code>	Numeric matrix of pilot expression (genes x samples). A <code>data.frame</code> is coerced to matrix.
<code>times</code>	Numeric vector of sample collection times in hours, length <code>ncol(data)</code> . The function does NOT validate units; pass hours (0 to 24 scale).
<code>period</code>	Period in the same units as <code>times</code> (default 24 hours).
<code>min_rhythm_pval</code>	Uncorrected cosinor p-value threshold for the rhythmic-candidate set (default 0.01). Used to compute <code>prop_rhythmic</code> and to select the top-K ($K = \min(\text{top_k}, n_{\text{rhythmic}})$) genes whose amplitudes, phases, and noise drive simulation truth.
<code>top_k</code>	Number of highest-signal genes (ranked by cosinor p-value) that form the effect-size distribution (amplitude, phase, noise). Default 300. If <code>top_k</code> exceeds the number of rhythmic genes, it is capped to that number with a warning. <code>Inf</code> or <code>"all"</code> uses every rhythmic candidate.
<code>prop_DR, prop_DP, prop_DM</code>	Proportions of differentially-rhythmic, differentially-phased, and differentially-mesor genes (defaults 0.15, 0.10, 0.00). Capped automatically at <code>prop_rhythmic</code> .

```
mesor_diff, phase_diff, amp_diff
```

Numeric length-2 ranges for the differential effect-size sampling.

```
dp_shift_mode
```

One of "fixed" or "uniform" (default "fixed").

```
dr_amp_scale, dr_sigma_scale
```

Scaling factors for DR-class amplitude and noise distributions (default 1).

```
paired_sigma
```

Logical, default FALSE. If TRUE, joint (A, sigma) sampling preserves pilot correlation in downstream simulations.

```
sim.seed
```

Integer seed for downstream simulation (default 12345).

```
verbose
```

Logical, default TRUE. Print summary at end.

Value

A `CircadianBioOptions` S3 object usable directly with `runSimsSingleCohort()` or `runDifferentialPower()`.

See Also

[estCircadianParamTwoGroup](#) for the two-group setup, [prepCircadianData](#) for preparing the input matrix.

Examples

```
set.seed(1)
n_genes <- 200; n_samples <- 30
times <- rep(seq(0, 22, by = 2), length.out = n_samples)
expr <- matrix(rnorm(n_genes * n_samples), nrow = n_genes)
for (g in 1:20) expr[g, ] <- expr[g, ] + 2 * cos(2 * pi * times / 24)
bio <- estCircadianParam(expr, times, period = 24,
                        top_k = 20, verbose = FALSE)
```

```
estCircadianParam2H
```

Two-Harmonic Circadian Pilot Estimation

Description

Two-harmonic analog of [estCircadianParam](#). Fits the regression

$$y = \mu + a_1 \cos(\omega_0 t) + b_1 \sin(\omega_0 t) + a_2 \cos(2\omega_0 t) + b_2 \sin(2\omega_0 t) + \epsilon$$

to each gene by OLS, tests the joint rhythmicity null $H_0 : a_1 = b_1 = a_2 = b_2 = 0$ on 4 d.f., and returns a `CircadianBioOptions` object whose 5-way joint empirical distribution $F_{(\mu, A_1, \phi_1, A_2, \phi_2, \sigma)}$ is preserved by gene-paired index draws downstream.

The returned object carries the standard 1-harmonic fields (`amplitude`, `phase`, `sigma_rhythmic`) holding the 1st-harmonic $A_{g,1}$, $\phi_{g,1}$, and σ_g respectively, and three extra fields attached to the list after construction:

- `amplitude2`: 2nd-harmonic amplitudes $A_{g,2}$
- `phase2`: 2nd-harmonic acrophases $\phi_{g,2}$ in hours $[0, period/2)$
- `paired_2h`: TRUE flag (downstream simulators read this to switch on the 2-harmonic generative model).

These extras live alongside `$diagnostics`; the constructor is not modified.

Usage

```
estCircadianParam2H(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 500,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

<code>data</code>	Numeric matrix of pilot expression (genes x samples).
<code>times</code>	Numeric vector of sample collection times in hours, length <code>ncol(data)</code> .
<code>period</code>	Period in the same units as <code>times</code> (default 24 hours).
<code>min_rhythm_pval</code>	P-value threshold for the joint 4-d.f. F-test (default 0.01). Genes with $p <$ this define the candidate set.
<code>top_k</code>	Cap on number of genes contributing to the joint $(A_1, \phi_1, A_2, \phi_2, \sigma)$ distribution (default 500), ranked by F-test p-value.
<code>prop_DR, prop_DP, prop_DM</code>	Differential proportions (defaults 0.15, 0.10, 0.00). Forwarded unchanged to <code>CircadianBioOptions</code> ; note the K=2 extension itself is single-cohort only.
<code>mesor_diff, phase_diff, amp_diff</code>	Differential effect-size ranges.
<code>dp_shift_mode</code>	"fixed" or "uniform" (default "fixed").
<code>dr_amp_scale, dr_sigma_scale</code>	DR scale factors (default 1).
<code>paired_sigma</code>	Logical (default FALSE). Forwarded to <code>CircadianBioOptions</code> .
<code>sim.seed</code>	Integer seed (default 12345).
<code>verbose</code>	Logical (default TRUE). Print diagnostic summary.

Value

A `CircadianBioOptions` object with extra fields `$amplitude2`, `$phase2`, `$paired_2h`, and a `$diagnostics` list summarising the 2H fit (median A2/A1 ratio, fraction of genes with significant 2nd-harmonic block, etc.).

Methodological notes

1. Pilot summary is a 5-way joint empirical distribution. When the simulator draws a rhythmic gene, the same index j is used across $(A_1, \phi_1, A_2, \phi_2, \sigma)$, preserving the cross-parameter dependence observed in the pilot.
2. No 2nd-harmonic significance threshold is applied: ALL top-K genes passing the joint 4-d.f. F-test contribute, even those whose A_2 is essentially noise. The empirical F_{A_2} will have a noise spike near zero, which is fine.
3. The K=2 extension is single-cohort only; the differential simulator is not extended.

See Also

[estCircadianParam](#) for the 1-harmonic case, [simCircadianSingleCohort2H](#) for the matching simulator.

Examples

```
# Two-harmonic pilot fit on a bundled raw pilot (first 300 genes).
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                           package = "SCP"))
expr <- praw$adrenal$expr[1:300, ]
bio2h <- estCircadianParam2H(expr, praw$adrenal$times,
                             top_k = 100, verbose = FALSE)
bio2h$prop_rhythmic
```

```
estCircadianParamTwoGroup
```

Two-Group Circadian Pilot Estimation

Description

Fits cosinor models to two groups of pilot data and assembles the two-group pilot summary used by the differential power simulator, encoding differential rhythmicity (DR), differential phase (DP), and differential MESOR (DM) structure.

Usage

```
estCircadianParamTwoGroup(
  data_1,
  data_2,
  times_1,
  times_2,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  phase_shift_threshold = 2,
  prop_DM = NULL,
  mesor_diff = NULL,
  dp_shift_mode = c("uniform", "fixed"),
  paired_sigma = FALSE,
  sim.seed = 12345,
```

```

    verbose = TRUE
  )

```

Arguments

<code>data_1, data_2</code>	Gene-by-sample expression matrices for the two groups.
<code>times_1, times_2</code>	Numeric collection times for each group.
<code>period</code>	Period in hours (default 24).
<code>min_rhythm_pval</code>	Marginal p-value screen defining the rhythmic pilot set.
<code>top_k</code>	Number of highest-signal genes (per group, ranked by cosinor p-value) forming each group's effect-size distribution (default 300). Capped (with a warning) to a group's rhythmic-gene count if larger; <code>Inf</code> or <code>"all"</code> uses every rhythmic candidate.
<code>phase_shift_threshold</code>	Minimum phase difference (hours) for the DP class.
<code>prop_DM</code>	Optional target proportion of differential-MESOR genes.
<code>mesor_diff</code>	Optional MESOR difference for the DM component.
<code>dp_shift_mode</code>	Differential-phase shift model, <code>"uniform"</code> or <code>"fixed"</code> .
<code>paired_sigma</code>	Preserve the paired (amplitude, noise) relationship across groups.
<code>sim.seed</code>	Random seed.
<code>verbose</code>	Print progress.

Value

A two-group `CircadianBioOptions` pilot summary.

Examples

```

# Two-group pilot (adrenal vs liver arms of the bundled demo pilot).
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                           package = "SCP"))
bio <- estCircadianParamTwoGroup(praw$adrenal$expr[1:300, ],
                                praw$liver$expr[1:300, ],
                                praw$adrenal$times, praw$liver$times,
                                top_k = 100, verbose = FALSE)

bio$prop_DR

```

Description

Converts raw expression data into the standard (matrix, times) format used by the pilot-calibration and power functions (for example `estCircadianParam` and `runBootstrapDesignGrid`).

The pipeline always expects:

- `data`, numeric matrix, **genes x samples**, log2-scale
- `times`, numeric vector, hours in [0, 24), one per column of data

This function handles the three most common raw formats:

"counts" Raw integer counts (e.g. `featureCounts`, `STAR`); converted to $\log_2(\text{CPM} + 1)$.

"cpm" Raw CPM values (e.g. `edgeR`, baboon data); converted to $\log_2(\text{CPM} + 1)$.

"log2" Already log2-scale (e.g. `GSE54651`, `Seney ACC`); wrapped with `data.matrix()`, no numeric transformation.

Usage

```
prepCircadianData(
  expr,
  times,
  input_type = c("counts", "cpm", "log2"),
  pheno = NULL,
  sample_col = NULL,
  time_col = NULL,
  period = 24,
  verbose = TRUE
)
```

Arguments

<code>expr</code>	A matrix, data.frame, or file path (CSV/TSV) of expression values. Rows = genes, columns = samples. Row names used as gene IDs.
<code>times</code>	Numeric vector of time-of-day values (hours, 0-24) for each sample, OR a character column name in <code>pheno</code> to extract times from.
<code>input_type</code>	One of "counts", "cpm", or "log2". Default "counts".
<code>pheno</code>	Optional data.frame of sample metadata. Must have the same column order as <code>expr</code> if <code>times</code> is a column name string.
<code>sample_col</code>	Optional: name of the column in <code>pheno</code> whose values match <code>colnames(expr)</code> for alignment. If NULL, assumes columns are already in the same order as rows of <code>pheno</code> .
<code>time_col</code>	Name of the TOD column in <code>pheno</code> (used when <code>times</code> is a column name string).
<code>period</code>	Circadian period in hours. Default 24. Used only for validation.
<code>verbose</code>	Print a short summary of what was done. Default TRUE.

Value

A list with:

`data` Numeric matrix (genes x samples), log2-scale

times	Numeric vector of hours (0-24), length == ncol(data)
n_genes	Number of genes
n_samples	Number of samples
input_type	The input_type used

Examples

```
# Wrap a bundled log2 pilot matrix into the (data, times) format.
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                           package = "SCP"))
expr <- praw$adrenal$expr[1:200, ]
tod <- praw$adrenal$times
prep <- prepCircadianData(expr, tod, input_type = "log2")
str(prepare(c("n_genes", "n_samples", "input_type")))
```

CircadianBioOptions

Create Biology and Differential Options

Description

Builds the `CircadianBioOptions` object that describes the simulated transcriptome: the number of genes, the proportion that are rhythmic, and the gene-level distributions of baseline expression, noise, amplitude, and phase. For two-group studies it also records the proportions of genes that differ in rhythmicity, phase, or mesor between groups. Most users obtain this object indirectly from `estCircadianParam` or `scp_load_pilot`; the constructor is exposed for building a configuration by hand.

Usage

```
CircadianBioOptions(
  ngenes = 5000,
  prop_rhythmic = NULL,
  period = 24,
  lBaselineExpr = NULL,
  lBaselineExpr2 = NULL,
  lod = NULL,
  lod2 = NULL,
  amplitude = NULL,
  amplitude2 = NULL,
  sigma_rhythmic = NULL,
  paired_sigma = FALSE,
  cts = NULL,
  cts2 = NULL,
  phase = "uniform",
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  mesor_diff = c(0.5, 2),
```

```

dp_shift_mode = c("fixed", "uniform"),
dr_amp_scale = 1,
dr_sigma_scale = 1,
alpha_pilot = NULL,
sim.seed = 12345
)

```

Arguments

<code>ngenes</code>	Number of genes.
<code>prop_rhythmic</code>	Proportion of rhythmic genes. If <code>NULL</code> and a pilot dataset name is supplied to <code>lBaselineExpr</code> , the pilot's estimated proportion is used.
<code>period</code>	Circadian period in hours.
<code>lBaselineExpr</code>	Log baseline expression. Can be a numeric scalar (constant for all genes), a numeric vector (resampled to <code>ngenes</code>), a function <code>f(n)</code> returning <code>n</code> values, or a character string naming a built-in pilot dataset stored in <code>data/</code> . No default, must be supplied.
<code>lBaselineExpr2</code>	As <code>lBaselineExpr</code> but for group 2 (differential analyses only). <code>NULL</code> uses the same distribution as group 1.
<code>lOD</code>	Log over-dispersion (noise). Same forms accepted as <code>lBaselineExpr</code> . No default, must be supplied.
<code>lOD2</code>	As <code>lOD</code> but for group 2. <code>NULL</code> shares group 1 values.
<code>amplitude</code>	Amplitude distribution for rhythmic genes, given in any of the forms accepted for <code>lBaselineExpr</code> . This argument is required and has no default.
<code>amplitude2</code>	As <code>amplitude</code> but for group 2.
<code>sigma_rhythmic</code>	Optional numeric vector of per-gene noise values for rhythmic genes (same length as <code>amplitude</code>). When provided, <code>amplitude</code> and <code>sigma</code> are drawn jointly to preserve pilot A-sigma correlation.
<code>paired_sigma</code>	Logical. If <code>TRUE</code> and <code>sigma_rhythmic</code> is supplied, expand <code>amplitude</code> and <code>sigma_rhythmic</code> jointly by the same resampled gene index so their pilot-estimated pairwise correlation is preserved; if <code>FALSE</code> (default), <code>amplitude</code> and <code>sigma</code> are expanded independently.
<code>cts</code>	Numeric vector of sample collection times for passive design.
<code>cts2</code>	As <code>cts</code> for group 2.
<code>phase</code>	Phase distribution for rhythmic genes. "uniform" (default) or the same forms as <code>lBaselineExpr</code> .
<code>prop_DR</code>	Proportion with differential rhythmicity.
<code>prop_DP</code>	Proportion with differential phase.
<code>prop_DM</code>	Proportion with differential mesor (mean shift, both groups rhythmic).
<code>phase_diff</code>	Range of phase shift for DP genes <code>c(min, max)</code> .
<code>amp_diff</code>	Range of amplitude ratio (unused; retained for interface compatibility).
<code>mesor_diff</code>	Range of mesor shift for DM genes <code>c(min, max)</code> (additive, log-scale units).

```

dp_shift_mode      "fixed" (use phase_diff[2]) or "uniform" (sample uniformly within
                    phase_diff range).
dr_amp_scale       Scale factor for amplitude (A) to adjust DR strength.
dr_sigma_scale     Scale factor for sigma to adjust DR strength.
alpha_pilot        Rhythmicity pre-screen threshold used to define the pilot's rhythmic gene set
                    (the value at which prop_rhythmic and the effect-size distribution were cal-
                    ibrated). Stored on the object for provenance and reporting; default NULL.
sim.seed           Random seed.

```

Value

Object of class "CircadianBioOptions".

Examples

```

# Build a synthetic biology configuration without pilot data.
bio <- CircadianBioOptions(ngenes = 300L, prop_rhythmic = 0.3,
                           lBaselineExpr = 5, lOD = -1, amplitude = 1)
bio

```

CircadianDesignOptions

Create Study Design Options

Description

Collects the sampling-design choices for a power analysis: the sample sizes to evaluate, the number of simulation replicates per size, and whether collection times are placed on an even grid (active design) or drawn from the pilot's time-of-day distribution (passive design). The resulting CircadianDesignOptions object is passed to the runSims* and recommendDesign functions.

Usage

```

CircadianDesignOptions(
  sample_sizes = c(10, 20, 40, 60, 80, 100),
  nsims = 100,
  design = c("active", "passive"),
  cts = NULL,
  B_values = NULL,
  test_types = c("DR", "DP", "DM")
)

```

Arguments

```

sample_sizes      Numeric vector of sample sizes to evaluate.
nsims              Number of simulation replicates run at each sample size.
design             Sampling design: "active" places collection times on an even grid, "passive"
                  draws them from cts.

```

<code>cts</code>	Numeric vector giving the time-of-day distribution used for a passive design.
<code>B_values</code>	Optional vector of time-point counts (B) to sweep, in addition to <code>sample_sizes</code> . NULL runs the single B implied by <code>cts</code> .
<code>test_types</code>	Character vector of differential tests to run, any of "DR", "DP", and "DM".

Value

Object of class "CircadianDesignOptions"

Examples

```
# Active design over two small sample sizes with few simulations.
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
dopt
```

CircadianAnalysisOptions

Create Analysis and Reporting Options

Description

Collects the settings that govern how each simulated dataset is analysed and how power is reported: the per-gene significance level, the multiple-testing correction, the FDR thresholds at which power curves are drawn, and the effect-size strata used to break power down by signal strength. The resulting `CircadianAnalysisOptions` object is passed to the `runSims*` and plotting functions.

Usage

```
CircadianAnalysisOptions(
  alpha = 0.05, p.adjust.method = "BH", parallel.ncores = 1,
  amp.cutoff = 0, target_effect = 0.1,
  fdr_thresholds = c(0.01, 0.05, 0.10, 0.20), reference_n = 60,
  r_strata = c(0, 0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75,
               2, 2.5, 3, 3.5, 4, 4.5, 5, Inf),
  strata_labels = NULL,
  phase_shifts = c(0, 0.5, 1, 2, 4, 6, 8, 10, 12),
  DCmethod = "DCP")
```

Arguments

<code>alpha</code>	Per-gene significance level for the cosinor rhythmicity test.
<code>p.adjust.method</code>	Multiple-testing correction applied across genes, passed to <code>stats::p.adjust</code> (default Benjamini-Hochberg).
<code>parallel.ncores</code>	Number of cores used to fit the per-gene tests.
<code>amp.cutoff</code>	Minimum amplitude a gene must exceed to enter the cosinor rhythmicity screen.
<code>target_effect</code>	Minimum effect size a gene must reach to count as a meaningful discovery.

<code>fdr_thresholds</code>	FDR thresholds at which power curves are reported.
<code>reference_n</code>	Reference sample size marked on the panel-B and phase-shift plots.
<code>r_strata</code>	Breakpoints, in $r = A/\sigma$ units, that define the effect-size strata used to report power by signal strength.
<code>strata_labels</code>	Labels for the strata; generated automatically from <code>r_strata</code> when NULL.
<code>phase_shifts</code>	Phase-shift magnitudes (hours) swept in the phase-sensitivity analysis.
<code>DCmethod</code>	Differential-rhythmicity detector to report against, either "DCP" (the cosinor F-test, default) or "CircaCompare". "DCP" is a retained legacy alias for the cosinor detector.

Value

Object of class "CircadianAnalysisOptions"

Examples

```
# Reporting options: 5 percent alpha, power at two FDR thresholds.
aopt <- CircadianAnalysisOptions(alpha = 0.05,
                                fdr_thresholds = c(0.05, 0.20))

aopt
```

CircadianBootstrapOptions
Create Bootstrap Grid Options

Description

Collects the settings for a bootstrapped design-grid analysis, which quantifies how much the finite pilot contributes to uncertainty in power across combinations of time points (B) and replicates (m). It defines the grid of designs to evaluate and the number of bootstrap draws and inner simulations used to form the confidence bands. The resulting object is passed to [runBootstrapDesignGrid](#).

Usage

```
CircadianBootstrapOptions(
  design_vector,
  B_values = c(4, 6, 8, 12, 24),
  m_values = NULL,
  N_values = NULL,
  nboot = 200,
  nsims_inner = 20,
  design = c("active", "passive"),
  seed = 42
)
```

Arguments

design_vector	Ordered vector of candidate time points to sample from; its length must be at least $\max(B_values)$.
B_values	Numbers of distinct time points to evaluate.
m_values	Replicates per time point. When NULL, derived as $\text{round}(N / B)$.
N_values	Total per-group sample sizes to evaluate. When NULL, derived as $B * m$.
nboot	Number of bootstrap parameter draws in the outer uncertainty loop.
nsims_inner	Number of simulations run per bootstrap draw.
design	Sampling design, either "active" or "passive".
seed	Random seed.

Value

Object of class "CircadianBootstrapOptions"

Examples

```
# A small active-design B-vs-m grid over two sample sizes.
bopt <- CircadianBootstrapOptions(design_vector = seq(0, 22, by = 2),
                                B_values = 4, N_values = c(24, 48),
                                nboot = 3, nsims_inner = 4)

bopt
```

```
runSimsSingleCohort
```

Single-Cohort Rhythmicity Power Simulation

Description

Extends the closed-form CircaPower approach by running full simulations: for each sample size and simulation replicate, generates data from the empirical pilot parameter distributions, applies the cosinor F-test per gene, applies BH correction, and aggregates empirical power and FDR.

Usage

```
runSimsSingleCohort(
  bio.opts,
  design.opts,
  analysis.opts,
  method = "cosinor",
  K = 1L,
  verbose = TRUE,
  mc.cores = 1L,
  progress = NULL
)
```

Arguments

<code>bio.opts</code>	The <code>CircadianBioOptions</code> pilot object, from <code>estCircadianParam()</code> .
<code>design.opts</code>	The <code>CircadianDesignOptions</code> object (sample sizes, nsims, design).
<code>analysis.opts</code>	The <code>CircadianAnalysisOptions</code> object (alpha, correction, FDR thresholds).
<code>method</code>	Detection method: "cosinor" (default, the K-harmonic cosinor F-test, with K set by the K argument), or the alternative detectors "JTK", "RAIN", and "MH" (multi-harmonic). "DCP" and "Kharmonic" are retained aliases of the cosinor detector.
<code>K</code>	Integer. Number of harmonics for the cosinor detector (default 1). K=1 is the single-harmonic cosinor F-test; K=2 captures the 1st and 2nd harmonics; K=3+ for very sharp peaks.
<code>verbose</code>	Print progress (default TRUE).
<code>mc.cores</code>	Parallel cores (default 1).
<code>progress</code>	Optional callback function(<code>sample_size_index</code> , <code>n_sample_sizes</code>) invoked after each sample size completes (used by the Shiny app to advance its progress bar). NULL (default) disables the callback.

Details

Simulation-based alternative to the closed-form `CircaPower` formula: works for any design (active or passive) and any pilot parameter distribution.

Value

List with:

<code>marginal_power</code>	Matrix [sample_sizes x nsims]
<code>marginal_FDR</code>	Matrix [sample_sizes x nsims]
<code>marginal_TD</code>	Matrix [sample_sizes x nsims]
<code>marginal_FD</code>	Matrix [sample_sizes x nsims]
<code>marginal_alpha</code>	Matrix [sample_sizes x nsims]: empirical type-I error
<code>pvalues</code>	Array [sample_sizes x ngenes x nsims]: raw p-values
<code>r_values_list</code>	List[[sample_size]][[sim]]: per-gene A/sigma
<code>strat_power</code>	Array [sample_sizes x n_strata x nsims]
<code>strat_TD</code>	Array [sample_sizes x n_strata x nsims]
<code>strat_FD</code>	Array [sample_sizes x n_strata x nsims]
<code>strat_n_targets</code>	Array [sample_sizes x n_strata x nsims]
<code>strat_n_nulls</code>	Array [sample_sizes x n_strata x nsims]
<code>strat_FDR</code>	Array [sample_sizes x n_strata x nsims]
<code>strat_alpha</code>	Array [sample_sizes x n_strata x nsims]

r_strata	Stratum breakpoints (from analysis.opts)
strata_labels	Stratum labels
n0_circapower	CircaPower n80 estimate from pilot median r
sample_sizes	Vector of sample sizes
nsims	Number of simulation replicates

Examples

```
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
bio$ngenex <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res <- runSimsSingleCohort(bio, dopt, aopt, verbose = FALSE)
rowMeans(res$marginal_power) # mean power at each N
```

```
runDifferentialPower
```

Run two-group differential circadian power analysis

Description

Estimates FDR-controlled power to detect differential rhythmicity between two groups. For each sample size and simulation replicate it generates paired two-group data from the pilot, applies the chosen differential detector, and aggregates power for the requested endpoints (differential rhythmicity, phase, or mesor). Unless `plot = FALSE`, it also draws the power curves and prints the recommended sample size on completion.

Usage

```
runDifferentialPower(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

bio.opts	Two-group pilot from <code>estCircadianParamTwoGroup()</code> .
design.opts	The <code>CircadianDesignOptions</code> object (sample sizes, design, collection times).

analysis.opts	The CircadianAnalysisOptions object.
methods	Detectors to run: "DCP" (the cosinor F-test) or "CircaCompare".
test_types	Endpoints to test, any of "DR" (rhythmicity), "DP" (phase), and "DM" (mesor). An endpoint a detector does not support is returned as NA.
alpha2	Scalar or vector swept for both groups
alpha3	Scalar or vector swept for both groups
mc.cores	Number of cores used for the per-gene fits.
plot	Draw the power curves when the run finishes.
output_file	Path for a PDF of the plot, or NULL to draw to the current device.
verbose	Print progress messages while running.

Value

A list of class `SCPDiffResult`. It holds the p-value and FDR arrays `pval_DR`, `fdr_DR`, `pval_DP`, `fdr_DP`, `pval_DM`, and `fdr_DM`, each indexed by gene, sample size, and simulation, together with `diff_type`, `effectsize`, `sample_sizes`, and `nsims`. Pass it to `plotDiffPower()` or to `npower(..., endpoint = "DR")`.

Examples

```

praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                           package = "SCP"))
e1 <- praw$adrenal$expr[1:300, ]; t1 <- praw$adrenal$times
e2 <- praw$liver$expr[1:300, ]; t2 <- praw$liver$times
bio <- estCircadianParamTwoGroup(e1, e2, t1, t2, top_k = 100,
                                verbose = FALSE)

bio$ngenes <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res <- runDifferentialPower(bio, dopt, aopt, test_types = "DR",
                          plot = FALSE, verbose = FALSE)
npower(res, endpoint = "DR", fdr = 0.05)$n

```

```
runBootstrapDesignGrid
```

Run bootstrap design grid simulation

Description

Estimates uncertainty in power across (N, B) design configurations by bootstrapping parameter sets from pilot data.

Usage

```
runBootstrapDesignGrid(
  pilot_data,
  pilot_times,
  boot.opts,
  analysis.opts,
  bio_diff.opts,
  pilot_data_2 = NULL,
  pilot_times_2 = NULL,
  mode = NULL,
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
  alpha3 = 0,
  min_rhythm_pval = 0.01,
  resample = c("subject", "gene"),
  verbose = TRUE,
  mc.cores = 1L
)
```

Arguments

pilot_data	Genes x samples matrix (pilot experiment).
pilot_times	Numeric vector of sample time points, length = ncol(pilot_data).
boot.opts	The bootstrap-grid options built with <code>CircadianBootstrapOptions()</code> . Must set <code>B_values</code> to a single value for passive designs.
analysis.opts	<code>CircadianAnalysisOptions</code> .
bio_diff.opts	<code>CircadianBioOptions</code> from <code>estCircadianParam()</code> . Required even in mode = "single", differential proportion fields are read from here.
pilot_data_2	Optional group-2 pilot matrix for mode = "differential".
pilot_times_2	Optional group-2 pilot times.
mode	"single" or "differential".
methods	Detection method(s): "DCP" (default), "JTK", "RAIN".
test_types	Endpoints for differential mode: any of "DR", "DP", "DM".
alpha2	Second-harmonic coefficient (default 0).
alpha3	Third-harmonic coefficient (default 0).
min_rhythm_pval	P-value threshold for pilot rhythmicity (default 0.01).
resample	Resampling scheme for the outer bootstrap. "subject" (default) is the Efron nonparametric bootstrap over pilot subjects: each draw resamples subject columns with replacement (carrying their collection times) and refits the pilot summary, matching the Section 2.3 description. "gene" is the legacy scheme that fits once and resamples gene rows of the fit; retained for backward compatibility and diagnostics only.
verbose	Print progress (default TRUE).
mc.cores	Parallel cores for inner simulation loop (default 1).

Value

Named list with 15 elements:

`N_values` Candidate sample sizes tested.
`B_values` Candidate time-point counts tested.
`m_matrix` Matrix of total samples ($N \times B$).
`nboot` Number of bootstrap replicates.
`design` Design type ("active" or "passive").
`test_types` Endpoints evaluated.
`fdr_threshold` FDR threshold used.
`boot_power` 4-D array [$nboot \times n_N \times n_B \times n_tests$].
`power_mean` Mean power array [$n_N \times n_B \times n_tests$].
`power_se` Bootstrap SE array, same shape as `power_mean`.
`power_ci_lo` 2.5th percentile CI array.
`power_ci_hi` 97.5th percentile CI array.
`optimal_B` Integer vector (length n_N): B with highest mean power at each N.
`optimal_B_ci_lo` Bootstrap 2.5th percentile of optimal B.
`optimal_B_ci_hi` Bootstrap 97.5th percentile of optimal B.

Examples

```
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                             package = "SCP"))
expr <- praw$adrenal$expr[1:300, ]
times <- praw$adrenal$times
bio <- estCircadianParam(expr, times, verbose = FALSE)
bopt <- CircadianBootstrapOptions(
  design_vector = sort(unique(round(times))),
  B_values = 4, N_values = c(24, 48),
  nboot = 3, nsims_inner = 4, design = "passive")
aopt <- CircadianAnalysisOptions(alpha = 0.05)
grid <- runBootstrapDesignGrid(expr, times, bopt, aopt,
                              bio_diff.opts = bio, mode = "single",
                              verbose = FALSE)

grid$power_mean
```

npower

Sample Size for a Target Genome-Wide Power

Description

Works on output from either `runSimsSingleCohort()` or `runSimsDiff()`. Recomputes marginal power at the requested FDR from raw simulation output, then returns the smallest grid N that meets the target (`n_grid`) and, when `interpolate = TRUE` (the default), a linearly interpolated estimate between the two bracketing grid points (`n`).

Usage

```
npower(
  res,
  target_power = 0.8,
  fdr = 0.05,
  endpoint = NULL,
  p.adjust.method = "BH",
  interpolate = TRUE
)
```

Arguments

<code>res</code>	Output from <code>runSimsSingleCohort()</code> or <code>runSimsDiff()</code> .
<code>target_power</code>	Numeric in (0, 1). Target marginal power (default 0.80).
<code>fdr</code>	FDR threshold (default 0.05).
<code>endpoint</code>	For <code>runSimsDiff()</code> output: one of "DR", "DP", "DM". Ignored for single-cohort output.
<code>p.adjust.method</code>	Multiple-testing correction (default "BH"). Only applied for single-cohort output (differential uses pre-computed FDR arrays).
<code>interpolate</code>	Logical. If TRUE (default), linearly interpolates between the two adjacent grid points to return a more precise <code>n</code> estimate. Set to FALSE to revert to the grid-snapped <code>n_grid</code> value (backward-compatible).

Value

An object of class "npower" (a list) with:

<code>n</code>	Recommended sample size: <code>n_interp</code> when <code>interpolate = TRUE</code> , otherwise <code>n_grid</code> . NA if target power is never reached.
<code>n_grid</code>	Smallest sample size in the simulation grid achieving target power, or NA.
<code>n_interp</code>	Linearly interpolated sample size between the two bracketing grid points (a fractional N rounded up). NA when <code>interpolate = FALSE</code> or when target power is never reached.
<code>power</code>	Named numeric vector: mean power at each grid sample size.
<code>target_power, fdr, endpoint</code>	Echo of inputs.

Examples

```
bio <- scp_load_pilot("human", "GTEX", "Adrenal", "All")
bio$ngenex <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res <- runSimsSingleCohort(bio, dopt, aopt, verbose = FALSE)
np <- npower(res, target_power = 0.8, fdr = 0.05)
np
```

```
circaPowerApproxN80
```

Analytical Sample Size for 80 percent Power

Description

Returns a fast closed-form estimate of the sample size needed for the target power, evaluated at the pilot's median effect size ($\tilde{r} = A/\sigma$). It uses the analytical cosinor F-test power formula rather than simulation, so it is a useful starting point before running the full simulation grid.

Usage

```
circaPowerApproxN80 (
  bio.opts,
  alpha = 0.05,
  target_power = 0.8,
  n_search = seq(5, 500, by = 5)
)
```

Arguments

<code>bio.opts</code>	A <code>CircadianBioOptions</code> pilot; its <code>amplitude</code> and <code>sigma_rhythmic</code> give the effect-size distribution.
<code>alpha</code>	Significance level for the per-gene test (default 0.05).
<code>target_power</code>	Target power to solve for (default 0.80).
<code>n_search</code>	Integer vector of candidate sample sizes to search over.

Value

A single integer: the smallest sample size in `n_search` that reaches `target_power`, or NA if none does.

Examples

```
# Analytical sample size for the pilot's median effect size.
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
circaPowerApproxN80(bio)
```

```
recommendDesign
```

B vs m design study (analytical and simulation)

Description

Compares study designs that trade the number of time points (B) against the number of replicates per time point (m), and recommends a sample size for a target power. It first reports how each detector responds to B versus m, then computes analytical power at each sample size for the cosinor detector (which is invariant to B), and finally runs the full simulation for detectors that have no closed form. A previous result can be supplied to reuse its simulation instead of running it again.

Usage

```
recommendDesign(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = c("DCP", "JTK", "RAIN", "MH"),
  target_power = 0.8,
  mode = c("single", "differential"),
  run_simulation = TRUE,
  prior_result = NULL,
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions from <code>estCircadianParam()</code> or <code>estCircadianParamTwoGroup()</code>
<code>design.opts</code>	CircadianDesignOptions with <code>sample_sizes</code> and <code>B_values</code>
<code>analysis.opts</code>	CircadianAnalysisOptions
<code>methods</code>	Detection methods to compare. Single-cohort: <code>c("DCP", "JTK", "RAIN", "MH")</code> . Differential: <code>c("DCP", "CircaCompare", "LimoRhyde", "DODR")</code> .
<code>target_power</code>	Target power for recommendation (default 0.80)
<code>mode</code>	"single" or "differential" (default "single")
<code>run_simulation</code>	TRUE = run full simulation after analytical step. FALSE = analytical (CircaPower) only.
<code>prior_result</code>	A previous <code>runSingleCohortGrid()/runDifferentialPower()</code> result to reuse instead of re-running simulation.
<code>alpha2</code>	2nd-harmonic deviation (scalar or vector)
<code>alpha3</code>	3rd-harmonic deviation (scalar or vector)
<code>mc.cores</code>	Parallel cores for simulation step
<code>plot</code>	Auto-plot final recommendation
<code>output_file</code>	PDF path (NULL = screen)
<code>verbose</code>	Print progress

Value

An `SCPRecommendResult` object (a list) with:

- `guidance`: the method-guidance table.
- `analytical_df`: analytical power at each sample size (cosinor detector only).
- `simulation`: the simulation result, or `NULL` when no simulation was run.
- `recommendation`: the recommended design and sample size per method.

Examples

```
# Analytical (CircaPower) design guidance, no simulation.
bio <- scp_load_pilot("human", "GTEX", "Adrenal", "All")
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48, 96), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
rec <- recommendDesign(bio, dopt, aopt, methods = "DCP",
                      run_simulation = FALSE, plot = FALSE,
                      verbose = FALSE)

rec$analytical_df
```

```
plotSingleCohortPower
```

Plot single-cohort circadian power (3 panels)

Description

Summarises a single-cohort power simulation in three panels: marginal power against sample size at several FDR thresholds (Panel A), the number of true discoveries by effect-size stratum (Panel B), and power resolved by effect-size stratum (Panel C). It takes the object returned by [runSimsSingleCohort](#) and either draws to the current device or writes a PDF.

Usage

```
plotSingleCohortPower(
  res,
  out_pdf = NULL,
  title = "",
  panel_a_res = NULL,
  panels = c("A", "B", "C"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  fdr = NULL,
  p.adjust.method = "BH",
  reference_n = NULL,
  display_sizes = NULL,
  r_max = 5,
  cex_main = 1.5,
  cex_lab = 1.6,
  cex_axis = 1.45,
  line_lwd = 2,
  pt_cex = 0.95,
  legend_cex = NULL,
  title_cex = 1.65,
  oma_top = 2.4,
  legend_pos = "topleft",
  se_cap = 0.3,
  vertical = FALSE,
  panel_c_legend = FALSE,
  width = 15,
```

```

    height = 5.5
)

```

Arguments

<code>res</code>	Output from <code>runSimsSingleCohort()</code> .
<code>out_pdf</code>	Path for PDF output. If NULL, plots to current device.
<code>title</code>	Overall figure title (default empty).
<code>panel_a_res</code>	Optional alternate result object used for the Panel A marginal-power computation only. If NULL (default), Panel A is computed from <code>res</code> . Useful when the marginal summary should reflect a different empirical assumption than the effect-size-stratified panels.
<code>panels</code>	Character vector, which of panels "A" (marginal power vs n), "B" (true discoveries by r-stratum), "C" (stratified power by r-stratum) to draw (default all three).
<code>fdr_thresholds</code>	FDR levels for panel A (default <code>c(0.01,0.05,0.10,0.20)</code>).
<code>panel_fdr</code>	FDR threshold highlighted in panels B/C (default 0.05).
<code>vline_power</code>	Horizontal reference line for target power (default 0.80).
<code>vline_fdr</code>	FDR threshold marked with a reference guide (default 0.20).
<code>fdr</code>	Single-value alias that sets both <code>panel_fdr</code> and <code>vline_fdr</code> at once (back-compat convenience; default NULL).
<code>p.adjust.method</code>	Correction method (default "BH").
<code>reference_n</code>	Reference sample size for annotations (default NULL = none).
<code>display_sizes</code>	Sample sizes to show in all panels (NULL = all).
<code>r_max</code>	Upper bound ($r = A/\sigma$ units) for the r-strata breakpoints; NULL adapts to the 95th percentile of observed r (default 5).
<code>cex_main</code>	Title character expansion (default 1.50).
<code>cex_lab</code>	Axis-label character expansion (default 1.60).
<code>cex_axis</code>	Axis-tick character expansion (default 1.45).
<code>line_lwd</code>	Line width for power curves (default 2).
<code>pt_cex</code>	Point character expansion for plotted markers (default 0.95).
<code>legend_cex</code>	Legend text character expansion; NULL (default) picks a size automatically.
<code>title_cex</code>	Outer-title character expansion (default 1.65).
<code>oma_top</code>	Outer margin (lines) reserved above the panels for the title (default 2.4).
<code>legend_pos</code>	Legend position keyword passed to <code>legend()</code> (default "topleft").
<code>se_cap</code>	Maximum half-width (power units) drawn for standard-error bars, to keep long bars from overplotting neighboring panels (default 0.3).
<code>vertical</code>	Logical; if TRUE, stack panels vertically instead of the default horizontal 1 x length(panels) layout (default FALSE).
<code>panel_c_legend</code>	Logical; if TRUE, also draw the legend on panel C (default FALSE; panel A's legend is normally sufficient).
<code>width</code>	PDF width in inches (default 15).
<code>height</code>	PDF height in inches (default 5.5).

Value

Invisibly returns list of data used per panel.

Examples

```
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
bio$ngenex <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res <- runSimsSingleCohort(bio, dopt, aopt, verbose = FALSE)
plotSingleCohortPower(res, out_pdf = tempfile(fileext = ".pdf"))
```

plotDiffPower

Plot differential circadian power

Description

Plots two-group differential power for one or more comparisons from the objects returned by [runDifferentialPower](#). By default it draws marginal power against sample size, with one column per endpoint and one row per comparison. Setting `stratified = TRUE` adds the effect-size-stratified panels (power and true discoveries by effect size) in a three-column layout.

Usage

```
plotDiffPower(
  res_list,
  comp_labels = names(res_list),
  endpoints = c("DR", "DP", "DM"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  display_sizes = NULL,
  stratified = FALSE,
  r_break_width = 0.25,
  r_max = 5,
  r_display_max = NULL,
  main_title = "Differential Circadian Power Analysis",
  out_pdf = NULL,
  width = NULL,
  height = NULL
)
```

Arguments

<code>res_list</code>	Named list of length 2, each element output of <code>runSimsDiff()</code> .
<code>comp_labels</code>	Character(2), display labels for the two comparisons.
<code>endpoints</code>	Which endpoints to include (default <code>c("DR", "DP", "DM")</code>).
<code>fdr_thresholds</code>	FDR levels for panel A (default <code>c(0.01, 0.05, 0.10, 0.20)</code>).

panel_fdr	FDR threshold used to draw the marginal power curve in the non-stratified layout (default 0.05).
vline_power	Horizontal reference line for target power, drawn at this value (default 0.80).
vline_fdr	FDR threshold marked with a reference guide on the power panels (default 0.20).
display_sizes	Sample sizes to show in all panels (NULL = all).
stratified	Logical (default FALSE). If FALSE, draws only the marginal power-vs-sample-size panel per endpoint (1 x n_endpoints layout per comparison). If TRUE, also draws power-by-r and true-discoveries-by-r strata panels in the original 3-column layout.
r_break_width	Width of r-strata bins (default 0.25). Ignored if stratified = FALSE.
r_max	Upper bound (in $r = A/\sigma$ units) for the r-strata breakpoints when stratified = TRUE (default 5).
r_display_max	Optional upper x-axis limit (r units) for the stratified panels; NULL (default) uses the full stratified range.
main_title	Outer plot title (default "Differential Circadian Power Analysis").
out_pdf	Path for PDF. If NULL, plots to current device.
width	PDF width in inches. Defaults adapt to stratified.
height	PDF height in inches. Defaults adapt to stratified.

Value

Invisibly, list of panel data per comparison/endpoint.

Examples

```

praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                             package = "SCP"))
e1 <- praw$adrenal$expr[1:300, ]; t1 <- praw$adrenal$times
e2 <- praw$liver$expr[1:300, ]; t2 <- praw$liver$times
bio <- estCircadianParamTwoGroup(e1, e2, t1, t2, top_k = 100,
                                verbose = FALSE)

bio$ngenes <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res <- runDifferentialPower(bio, dopt, aopt, test_types = "DR",
                           plot = FALSE, verbose = FALSE)
plotDiffPower(list(res), endpoints = "DR",
               out_pdf = tempfile(fileext = ".pdf"))

```

```
plotBootstrapDesignGrid
```

Plot bootstrap design grid results

Description

Panel A: Power vs N, one line per B, with bootstrap CI ribbon. Panel B: Heatmap of mean power over (N, B) grid.

Usage

```
plotBootstrapDesignGrid(
  result,
  test_type = "DR",
  fdr_threshold = NULL,
  panels = "A",
  output_file = NULL
)
```

Arguments

<code>result</code>	Output from <code>runBootstrapDesignGrid()</code>
<code>test_type</code>	Test type to plot
<code>fdr_threshold</code>	FDR threshold (label only)
<code>panels</code>	Which panel(s) to draw: "A" (power vs N) and/or "B" (heatmap) (default "A").
<code>output_file</code>	Path for PDF output (NULL = screen)

Examples

```
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                             package = "SCP"))
expr <- praw$adrenal$expr[1:300, ]
times <- praw$adrenal$times
bio <- estCircadianParam(expr, times, verbose = FALSE)
bopt <- CircadianBootstrapOptions(
  design_vector = sort(unique(round(times))),
  B_values = 4, N_values = c(24, 48),
  nboot = 3, nsims_inner = 4, design = "passive")
aopt <- CircadianAnalysisOptions(alpha = 0.05)
grid <- runBootstrapDesignGrid(expr, times, bopt, aopt,
                              bio_diff.opts = bio, mode = "single",
                              verbose = FALSE)
plotBootstrapDesignGrid(grid, test_type = "rhythmic",
                        output_file = tempfile(fileext = ".pdf"))
```

detect_cosinor	<i>Unified cosinor rhythmicity detection (single- or multi-harmonic)</i>
----------------	--

Description

Canonical entry point for rhythmicity detection by K-harmonic cosinor regression. $K = 1$ is the single-harmonic cosinor F-test, identical to the single-harmonic case; $K = 2$ adds the 12-hour second harmonic and tests the joint harmonic block. By default the unshrunk exact nested F-test is used (matching the power derivations in the paper). Returns raw p-values; the caller applies FDR control.

Usage

```
detect_cosinor(expr, times, K = 1L, period = 24,
               ebayes = FALSE, mc.cores = 1L)
```

Arguments

expr	Gene x sample expression matrix
times	Numeric time vector (length = ncol(expr))
K	Harmonic order (default 1)
period	Period in hours (default 24)
ebayes	For $K \geq 2$, moderate residual variance with empirical-Bayes shrinkage (default FALSE, the unshrunk exact F-test)
mc.cores	Cores for the $K \geq 2$ fit (default 1)

Value

Numeric vector of raw p-values, length `nrow(expr)`

See Also

[estCircadianParam](#) for calibrating a pilot from the same cosinor fit.

Examples

```
# Rhythmicity p-values for a small simulated matrix (10 rhythmic genes).
set.seed(1)
times <- rep(seq(0, 22, by = 2), 2)
expr <- matrix(rnorm(50 * length(times)), nrow = 50)
expr[1:10, ] <- expr[1:10, ] + 2 * cos(2 * pi * times / 24)
pvals <- detect_cosinor(expr, times, K = 1)
sum(p.adjust(pvals, "BH") < 0.05)
```

simCircadianSingleCohort2H

Simulate Two-Harmonic Single-Cohort Data

Description

Two-harmonic analog of the single-harmonic single-cohort simulator. Generates expression from

$$y_{gi} = \mu_g + A_{g,1} \cos(\omega_0(t_i - \phi_{g,1})) + A_{g,2} \cos(2\omega_0(t_i - \phi_{g,2})) + \epsilon_{gi}$$

using the joint pilot tuple $(A_1, \phi_1, A_2, \phi_2, \sigma)$ stored on `bio.opts` by [estCircadianParam2H](#). The same index is drawn across all five vectors so the pilot's cross-parameter dependence is preserved.

`r_values` is reported on the 1st-harmonic scale $r_g = A_{g,1}/\sigma_g$ to match the existing r-stratification machinery and the manuscript's r_g notation.

Usage

```
simCircadianSingleCohort2H(bio.opts, cts, seed = NULL)
```

Arguments

<code>bio.opts</code>	A <code>CircadianBioOptions</code> produced by estCircadianParam2H (must carry <code>paired_2h = TRUE</code> , <code>amplitude2</code> , <code>phase2</code>).
<code>cts</code>	Numeric vector of sample collection times in hours; length determines N .
<code>seed</code>	Optional integer random seed.

Value

Named list:

`expr` Gene expression matrix ($n_{\text{genes}} \times N$).

`is_rhythmic` Logical vector.

`r_values` Per-gene 1st-harmonic SNR $A_{g,1}/\sigma_g$.

`A1, phi1, A2, phi2` Per-gene truth vectors (length n_{genes} ; zero for non-rhythmic genes).

See Also

[estCircadianParam2H](#) for the matching pilot fit.

Examples

```
# Generate one two-harmonic dataset from a 2H pilot fit.
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                             package = "SCP"))
expr <- praw$adrenal$expr[1:300, ]
bio2h <- estCircadianParam2H(expr, praw$adrenal$times,
                             top_k = 100, verbose = FALSE)

bio2h$ngenes <- 300L
cts <- rep(seq(0, 22, by = 2), 2)
sim <- simCircadianSingleCohort2H(bio2h, cts, seed = 1)
dim(sim$expr)
```

```
makeAdaptiveRStrata
```

Adaptive r-Strata Breakpoints

Description

Generates breakpoints from 0 to $\text{ceiling}(r_{\text{max}} / \text{bin_width}) * \text{bin_width}$, stepping by `bin_width`, then appends `Inf`. Every bin width is identical, but the number of bins adapts to the empirical `r` range of the pilot data.

Usage

```
makeAdaptiveRStrata(
  bio.opts,
  bin_width = 0.25,
  r_min_pct = 0,
  r_pctile_cap = 0.99
)
```

Arguments

<code>bio.opts</code>	CircadianBioOptions with <code>amplitude</code> and <code>sigma_rhythmic</code> .
<code>bin_width</code>	Width of each <code>r</code> bin (default 0.25).
<code>r_min_pct</code>	Lower percentile of pilot <code>r</code> used as range floor (default 0 = always start from 0).
<code>r_pctile_cap</code>	Upper percentile of pilot <code>r</code> used as the range ceiling before rounding up to the nearest <code>bin_width</code> (default 0.99).

Value

Numeric vector of breakpoints starting at 0 and ending at `Inf`.

Examples

```
# Adaptive r = A/sigma strata from a bundled pilot.
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
makeAdaptiveRStrata(bio, bin_width = 0.5)
```

```
launchShiny
```

Launch the SCP Shiny app

Description

Opens the bundled simulation-based power-analysis app in the user's default browser. The app lets the user pick a bundled pilot (or upload their own), set target power and FDR, read off the recommended sample size, explore per-gene rhythmicity, and run pathway enrichment.

Usage

```
launchShiny(..., install_deps = TRUE, species_all = FALSE)
```

Arguments

<code>...</code>	Additional arguments passed to <code>shiny::runApp()</code> .
<code>install_deps</code>	Logical; if <code>TRUE</code> (default) install any missing optional app packages before launching. Set <code>FALSE</code> to skip the check (missing features then degrade gracefully inside the app).
<code>species_all</code>	Logical; if <code>TRUE</code> also install the annotation databases for rat, zebrafish, <i>Drosophila</i> , and <i>Arabidopsis</i> (in addition to human + mouse) so upload gene-symbol mapping works for every supported species. Default <code>FALSE</code> installs only human + mouse (the common case); the others install on demand or you can pass <code>TRUE</code> once.

Details

On first launch the function checks for the app's optional helper packages and offers to install any that are missing (so the gene-symbol mapping, XLSX export, full-app capture, and pathway-enrichment features work out of the box).

Value

Invisibly returns `NULL`. Called for its side effect of launching the Shiny app.

Examples

```
## Not run:
  launchShiny()
  launchShiny(species_all = TRUE)    # add rat/fish/fly/plant DBs
  launchShiny(install_deps = FALSE) # skip the dependency check

## End(Not run)
```

Index

CircadianAnalysisOptions, [1](#), [15](#)
CircadianBioOptions, [1](#), [12](#)
CircadianBootstrapOptions, [1](#), [16](#)
CircadianDesignOptions, [1](#), [14](#)
circaPowerApproxN80, [1](#), [24](#)

detect_cosinor, [2](#), [31](#)

estCircadianParam, [1](#), [4](#), [6](#), [7](#), [9](#), [12](#), [31](#)
estCircadianParam2H, [1](#), [7](#), [32](#)
estCircadianParamTwoGroup, [1](#), [7](#), [9](#)

launchShiny, [2](#), [33](#)

makeAdaptiveRStrata, [2](#), [33](#)

npower, [1](#), [22](#)

plotBootstrapDesignGrid, [2](#), [30](#)
plotDiffPower, [2](#), [28](#)
plotSingleCohortPower, [2](#), [26](#)
prepCircadianData, [1](#), [7](#), [10](#)

recommendDesign, [2](#), [24](#)
runBootstrapDesignGrid, [1](#), [16](#), [20](#)
runDifferentialPower, [1](#), [19](#), [28](#)
runSimsSingleCohort, [1](#), [17](#), [26](#)

SCP (*SCP-package*), [1](#)
SCP-package, [1](#)
scp_load_pilot, [1](#), [2](#), [3](#), [12](#)
scp_pilot_database, [2](#)
scp_pilot_search, [1](#), [2](#), [5](#)
scp_pilots, [1](#), [2](#), [3](#)
simCircadianSingleCohort2H, [2](#), [9](#),
[32](#)